第四次考核前導練習

怎麼讓 agent 做事，並證明它做對了

```
$ ./verify/checkout.sh
$ echo $?
0
```

BIRC ● 115 年 11 月 ● 90 分鐘

第一節 定錨 · 5 min

agent 是什麼？有哪些工具？

agent＝一個迴圈　讀專案 → 想 → **真的執行指令、改檔案** → 看結果 → 再想再改，直到它認為完成。

你給任務與完成條件，它在你的機器上改真的檔案——所以接下來整堂課只有兩個主題：**驗證**、**文件**。

終端機　Claude Code

Anthropic 官方；終端機 agent 標竿，教學用它最多

終端機　Codex

OpenAI 官方 CLI（也有雲端版）；GPT 家族、沙箱化

開源　opencode

開源終端機 TUI；接任何模型、build／plan 雙模式

開源　DeepSeek Harness

DeepSeek 官方開源；「一切皆插件」架構、獨立配置與沙箱、會話可回放

開源　omo

多模型編排：任務拆給多個模型／子任務協調執行

IDE　Antigravity

Google 的 agentic IDE；多 agent 並行、內建瀏覽器

助理型　Hermes

自改進型：經驗沉澱成技能、有持久記憶

助理型　OpenClaw

自己機器上的個人助理；可經通訊工具操作電腦

自選，正式考不限工具　下節五個動作任何工具都一樣——今天就用正式考要用的那個。

第一節 定錨 · 5 min

正式考，考什麼？

分數落在**可觀察的產出**，不在程式碼漂不漂亮；**五份交付物**都要在 repo 裡。

① 定義問題與規格

`SPEC.md 20 分`

`AGENTS.md 5 分`

業務規則表＋API 一覽＝評審的驗收標準

② 測試與評估（evals）

`verify/ 15 分`

九個 API 各一支腳本，乾淨環境重跑全綠

③ 領域知識與判斷

`30 分`

併發安全、/chat 串 BIRC-LLM、decisions.md

④ 為結果負責

`15 分`

audit.md（錯什麼、怎麼發現、怎麼修）＋ Live demo

⑤ 驗證

`40 分`

隱藏驗收、評審抽跑 verify/、10 分鐘口試

五份交付物 `SPEC.md` `AGENTS.md` `verify/` `audit.md` `decisions.md`

缺任一項，對應項目歸零；另有必做創意題 15 分。今天的核心產出一樣：一筆 agent 失敗——用自己的 verify 抓到、自己修好、自己寫進 audit。

第一節 定錨 · 5 min

今天的規則＝正式考的規則

① 可以帶模板

自己準備的 prompt、`AGENTS.md` 模板，還有今天發的 SPEC 骨架、範例腳本，都能帶進場。

② agent 工具自選

用哪個都可以。

今天就用你正式考要用的那個，手感才對。

③ 結尾同伴互講

每個人都要對旁邊的人講 1 分鐘。

agent 哪裡錯、你怎麼抓到的。

今天的兩條主線　**驗證**——別信 agent 的嘴，信 exit 0；**長記憶**——agent 不記得任何事，專案裡的文件才記得。

開始前　現在就把你的模板、agent 工具準備好——前導練習照正式考的規則走。

第一節 定錨 · 5 min

90 分鐘總覽

0–5	**定錨＋第一次：直接實作**	聽＋開工
5–20	**agent 基本操作 15 min**	考生做
20–28	**agent 會搞砸 ×4**	現場演・最重要
28–48	**帶著寫 SPEC**	你寫
48–70	**第二次實作：新專案**	考生做
70–90	**審計＋收束**	考生做＋講

T+48 SPEC 缺口補齊　　T+55 一起跑併發 case　　T+90 硬截止

第一節 定錨 · 5 min

前導題目：借出＋歸還——第一次，直接實作

正式考 9 個 API，今天只做 2 個

`POST` `/loans/{id}/checkout`

`POST` `/loans/{id}/return`

- seed 已有 1 筆 `RESERVED` ＋對應器材
- 不用實作預約
- 業務規則只給三條粗的

practice_pkg＝今天的戰場

- 能跑的 skeleton＋SQLite＋seed
- `SPEC.md` 兩處 TBD　`AGENTS.md` 四項＝完成標準
- `verify/` 2 支範例腳本＋ `reset.sh`
- 1 筆有提示的髒資料： `dueAt` 早於 `lentOutAt`

現在做的事：開 agent → 直接實作（TBD 先不填、verify 先不寫）→ 看它哪裡錯

第一次：只給專案，直接實作
SPEC 骨架的兩處空白：**TBD-1** 併發處理機制（你自己選）　**TBD-2** 錯誤回應契約——第三節再填。

第二節 基本操作 + agent 會搞砸 · 23 min

15 分鐘上手：不管你用哪個 agent

開 在 `practice_pkg/`（專案根目錄）啟動 agent。

下指令 給**有明確完成條件**的小任務：「列出專案裡的例外類別」。別丟一句模糊的「幫我看看專案」。

看改動 `git diff` 看**實際**改動。agent 的總結是 PR 描述，**diff 才是真相**。

叫它跑 讓它跑 verify 腳本，你自己看輸出。**exit 0 是唯一可信訊號**。

退回 改壞就中斷它，用 git 退回。改壞不丟人，**不知道改壞了**才丟人。

沒裝 `birc`（骨架碼產生工具）？

```
npm i -g birc-generator
```

更新到最新：

```
birc update
```

開始實做 **15 分鐘計時**——開 agent，把上面五個動作各做一遍。

規格沒寫死 → agent 靜默挑一種解讀

觸發 下含糊指示：「錯誤的時候回個錯誤訊息就好」→ agent 自信地**發明**一套錯誤格式（狀態碼、回應體都自己編）。**規格空白＝授權 agent 猜。**

怎麼改 告訴它真正的契約：「SPEC 寫 409＋ `Result` 外殼，照這個改」→ 用 `git diff` 看它改對沒有。

怎麼防 `SPEC.md` 的**錯誤回應契約**：狀態碼＋回應體格式寫死。寫進文件，agent 就不用每次都猜。

怎麼驗 錯誤 case（負例）寫進 verify：斷言**狀態碼＋回應體**，重跑看 exit code。

說完成了，其實是錯的

觸發 agent 回報「實作完成、跑過、全綠」→ **你自己重跑 verify**：有一支 case 紅，或 DB 狀態不是該有的值。

怎麼改 指著失敗的 case 叫它修，重跑直到過；改太多就用 `git` 退回、重下指令。

怎麼防 完成標準**不是「它說完成」，是「verify 過」**——唯一可信的完成訊號：你的 verify 腳本 **exit 0**。

怎麼驗 verify 要**可重跑＋exit code＋負例**，三缺一就不叫 eval；要能從乾淨環境重跑。

併發裸 UPDATE

觸發 SPEC 故意不寫併發；當場跑併發 case → **數量扣成負數、同一件被借兩次**。

怎麼改 叫它重讀你指定的 TBD-1 機制，重寫；重跑併發 case 到**恰好 1 筆成功**。

怎麼防 **TBD-1** 指定機制：唯一約束／樂觀鎖／悲觀鎖／其他，擇一，寫進 SPEC。

怎麼驗 併發 case 寫進 `verify/` 必跑：同時借出同一器材，斷言**恰好 1 筆成功、qty ≥ 0**。不跑併發測試，你根本看不見這個 bug。



改了程式，沒改文件

觸發	agent 改完行為，問它「文件更新了嗎？」→ 沒。AGENTS.md／SPEC 還是**舊規格**。
怎麼改	當場叫它補文件；你自己查一遍**程式 ↔ 文件一致**。
怎麼防	AGENTS.md 寫死「**行為改了，文件同一個 PR 更新**」——agent 的長記憶就是專案文件：**沒寫下來的，就不存在**。
怎麼驗	收尾前問它「讀文件，告訴我 X 現在的規則」，跟程式對——對不上就是文件過時。

第二節 基本操作＋agent 會搞砸 · 23 min

四件「agent 會搞砸」，都照這三步走

① 觸發失敗

現場觸發，**活的**，不放截圖。

② 怎麼改・指出扣分點

現場改、指出扣分——這裡**就是**正式考會扣你分的地方。

③ 怎麼防・怎麼驗

SPEC 一句話／verify 一個 case／decisions 一筆。

每個 2 分鐘

第三節 帶著寫 SPEC · 20 min

帶著填 SPEC：要學的是「問法」

> **如果你是 agent，讀到這份規格會有幾種做法？**
> **你允許幾種？**

第一步 業務規則用白話寫：「狀態不是 RESERVED 就不能借出」。先不碰程式。

第二步 枚舉解讀：如果你是 agent，這句話你有幾種做法？會錯在哪？

第三步 收斂成條款：一條規則一句話，邊界與錯誤契約寫進 SPEC。

- **TBD-2**：給範例格式，照抄改寫即可。
- **TBD-1**：只要求「機制必須指定、寫進 SPEC」，選哪個你自己決定。

T+48 硬時間點　SPEC 兩段都要寫好——**不准帶著漏洞進實作**。

第四節 實作 · 22 min

第二次實作：新專案、同一題

開一個全新的 practice_pkg，再實作一次借出＋歸還——這次 SPEC 寫好了、AGENTS.md 寫好了、verify 備好了。

看跟第一次差在哪　有文件＋驗證，agent 少問、你少追、**verify 當場抓**。

對 agent 下工單　你的手不離開 verify 腳本。agent 在跑 ≠ 你可以空等：**你的 verify 腳本，現在就在寫**。

T+55：大家一起跑併發 case

有人失敗　當場走一輪：**發現 → 定位 → 修 → 複跑 → 記 audit**

沒有人給答案——case 是你自己的。

沒人失敗　當場抽一份 TBD-1 填法：「**僥倖通過**」和「**真正安全**」差在哪。

第五節 審計 · 20 min

審計：複跑＋記錄

① verify 全套複跑

評審會抽跑，先練一次

② 寫 audit.md

1 筆，四要素齊全

③ 寫 decisions.md

併發那筆＋**否決的方案**

```
audit.md
## A-01 agent 寫的分頁查詢沒限制 size 上限
- 現象：agent 哪裡錯、錯成什麼樣
- 證據：哪支腳本、哪個 case、看到什麼輸出（verify 輸出或 transcript 片段）
- 修法：改了什麼
- 防再犯：補了哪個 verify case／哪句 SPEC
```

「證據」要指向　具體的腳本和 case。最容易編的一欄，也最容易被看穿。

第六節 收束 · 5 min

收束：一分鐘口頭陳述

兩兩一組，每人對旁邊的人講 1 分鐘

agent 哪裡錯？
我怎麼抓到的？

口試暖身，不計分。

聽的人負責追問；老師也會問沒教過的點，例如：

「評審多跑一次你的 verify，會怎樣？」

講的時候也想一想　第一次（直接實作）vs 第二次（文件＋驗證），差在哪——那就是**為什麼要驗證、為什麼要文件**的答案。

第六節 收束 · 5 min

今天 vs. 正式考

	今天前導	正式考
API	2 個	**9 個**
時間	90 分鐘	**180 分鐘**
SPEC 缺口	2 處・有提示	**自己補完整・無提示**
帶領	老師帶前半	**全程沒人帶**
髒資料	1 筆・有提示	**2 筆・無提示**
併發腳本	老師提供	**自己寫**
audit	≥ 1 筆	**≥ 3 筆**
驗收	老師追問	**評審抽跑＋口試**
`/chat`	不做	**必做**

必做：機器人問答 `POST /chat`

- 必須串中心的 **BIRC-LLM**
- 只回答**查詢類**問題
- 先查自己系統的真實資料，再交給 LLM
- 查無根據 → 回 SPEC 定的固定訊息
- LLM 掛掉**不得回 500**

第六節 收束 · 參考頁

機器人問答 /chat：怎麼串 BIRC-LLM

BIRC-LLM＝中心提供的 OpenAI 相容端點

- 三個參數：`VLLM_BASE_URL` ／ `VLLM_API_KEY` ／ `VLLM_MODEL=BIRC-LLM`
- URL 與 key 考前提供；key 放環境變數，不寫進程式、不 commit
- 最快通連：`npx @lucashsu95/ai-forge init . --yes`
- 再跑 `ai-forge doctor` 依序查金鑰 → 端點 → 模型

```
BIRC-LLM · OpenAI 相容 · POST /chat/completions
Authorization: Bearer {VLLM_API_KEY}
{ "model": "BIRC-LLM", "messages": [
  { "role": "system", "content": "只用我給的資料回答；查不到就回答：SPEC 定的固定訊息" },
  { "role": "user", "content": "你查 DB 得到的資料＋會員的問題" }
] }
```

資料流　先查**自己系統**的真實資料 → 餵 LLM → 回文字。**不讓 LLM 憑自己的知識作答。**

LLM 掛掉／逾時　回你在 SPEC 定的狀態碼＋訊息——**不得回 500**。